

FastAPI Engineering Standards & Architecture Team Guide

Modular Monolith · Scalable Architecture · Microservice-Ready

01 Project Structure	02 Core Conventions
03 Module Standards	04 Auth & Security
05 Background Workers	06 Adding New Modules
07 Scalability	08 Microservice Extraction
09 Strict Rules	

01 — Introduction & Purpose

This document is the official FastAPI engineering standard for all projects built at Sambhava AI & Tech Services. Every developer — frontend, backend, DevOps, or intern — must read and follow this guide before writing a single line of backend code. This is not a suggestion; it is the contract we operate under.

The architecture described here follows a **Modular Monolith** pattern — one deployable unit, internally structured with strict vertical domain boundaries. Each domain module is fully self-contained, meaning it can be extracted into an independent microservice in the future with minimal refactoring.

Core Goals of This Standard

- Any developer can pick up any module and understand it within minutes
- No knowledge is locked in a single person's head
- Business domains are isolated — changes in one never break another
- Auth, security, and encryption are enforced uniformly across all projects
- The app can scale horizontally and extract to microservices without rewrite

FastAPI	PostgreSQL	Redis	ARQ Workers	Docker	Alembic
---------	------------	-------	-------------	--------	---------

02 — Complete Folder Structure

The following structure is the **non-negotiable template** for every new project. Do not rename folders, do not merge layers, do not skip files. This is the structure your team clones from the Sambhava boilerplate repository.

```
project-name/
├──
├── app/
│   ├── main.py           # FastAPI app init, middleware registration
│   ├── config.py         # All env vars via pydantic-settings
│   ├── dependencies.py   # Shared FastAPI dependencies
│   └──
│   ├── core/             # ★ Foundation – read this first
│   │   ├── security.py   # JWT creation, verification, hashing
│   │   ├── encryption.py # AES-256-GCM field-level encrypt/decrypt
│   │   ├── exceptions.py # Custom exception classes
│   │   ├── responses.py  # Unified API response wrapper
│   │   └── logging.py    # Structured logging setup
│   └──
│   ├── db/               # DB connections – never in modules
│   │   ├── postgres.py   # SQLAlchemy async engine + session
│   │   ├── mongo.py      # Motor async client (if used)
│   │   └── redis.py      # Redis client – auth + cache
│   └──
│   ├── middleware/
│   │   ├── auth.py       # Token validation
│   │   ├── rate_limiter.py # Per-route rate limiting
│   │   └── request_logger.py
```

```
■   ■■■ modules/                # ★ All business logic lives here
■   ■   ■■■ auth/
■   ■   ■   ■■■ router.py      # HTTP endpoints only
■   ■   ■   ■■■ service.py     # Business logic
■   ■   ■   ■■■ repository.py
■   ■   ■   ■■■ schemas.py     # Pydantic request/response models
■   ■   ■   ■■■ models.py      # SQLAlchemy models
■   ■   ■
■   ■   ■■■ users/             # Same 5-file structure
■   ■   ■■■ notifications/
■   ■   ■   ■■■ providers/     # Provider adapters
■   ■   ■       ■■■ base.py    # Abstract interface
■   ■   ■       ■■■ fcm.py
■   ■   ■       ■■■ email.py
■   ■   ■       ■■■ sms.py
■   ■   ■■■ [your-domain]/     # Project-specific modules
■   ■
■   ■■■ workers/               # ARQ background task worker
■       ■■■ arq_settings.py
■       ■■■ tasks/
■           ■■■ notification_tasks.py
■           ■■■ email_tasks.py
■
■■■ migrations/                # Alembic – never edit manually
■■■ tests/
■■■ scripts/
■   ■■■ seed.py
■   ■■■ create_module.py       # ★ Scaffold new modules via CLI
■■■ .env.example
■■■ docker-compose.yml
■■■ Dockerfile
■■■ Dockerfile.worker         # Separate ARQ worker image
■■■ pyproject.toml
■■■ README.md
```

03 — Layer Responsibilities

Every module contains exactly five files. Each file has a strict, non-overlapping responsibility. Mixing responsibilities across files is a code review rejection reason.

router.py — HTTP only

Defines FastAPI routes. Validates request via schema. Calls service. Returns response. Zero business logic. Zero database access. If you write an if/else here that isn't about HTTP status, move it to service.

service.py — Business Logic

Orchestrates the workflow. Calls repository for data. Calls other modules' services (never their repositories). Raises domain exceptions. Dispatches worker tasks. This is where all decision-making lives.

repository.py — Data Access

Only constructs and executes database queries. No business logic. No if/else based on data values. Returns raw ORM objects or dicts. Service layer decides what to do with results.

schemas.py — Pydantic Models

Request body models, response models, internal DTOs. All field-level validation here. Never import SQLAlchemy models into schemas. Schemas are the public contract of the module.

models.py — DB Models

SQLAlchemy ORM table definitions. No methods with business logic. Foreign keys can reference other modules' tables but via string references only — never import another module's model class directly.

The Golden Rule — Never Skip Layers

Router → Service → Repository → Database

- Router must never call repository directly
- Repository must never contain if/else business logic
- Service must never import another module's repository

04 — Core Layer Deep Dive

The core/ folder is the foundation of the entire application. It is shared across all modules and all projects. Never put business logic here. These are pure utilities that modules consume.

4.1 — Authentication: JWT + Refresh Token + Redis Blacklist

Access tokens are short-lived (15 minutes). Refresh tokens are long-lived (7–30 days) stored in Redis with rotation. On every refresh, the old token is invalidated and a new pair is issued. If a stolen token is used, rotation detection invalidates both sides immediately.

Access Token TTL	15 minutes — signed JWT, stateless, fast
Refresh Token TTL	7 days (configurable per project)
Storage	Refresh token stored in Redis with user ID as key
Blacklist	Logged-out tokens added to Redis blacklist set
Algorithm	HS256 minimum, RS256 recommended for multi-service
Password Hashing	bcrypt with cost factor 12

4.2 — Field-Level Encryption: AES-256-GCM

Full request/response encryption is avoided — it makes logging, debugging, and monitoring impossible. Instead, sensitive fields (Aadhaar, PAN, financial data, personal identifiers) are encrypted at the field level before DB storage using AES-256-GCM. The encryption key is loaded from environment, never hardcoded.

```
# core/encryption.py — standard usage across all projects
from core.encryption import encrypt_field, decrypt_field

# Before saving to DB
user.aadhaar = encrypt_field(raw_aadhaar)

# Before returning in response
response.aadhaar = decrypt_field(user.aadhaar)
```

4.3 — Unified Response Wrapper

Every API response must use the standard wrapper from core/responses.py. This ensures all clients get a consistent structure regardless of which module or developer wrote the endpoint.

```
# Every successful response
{"success": true, "data": {...}, "message": "User created", "code": 201}

# Every error response
{"success": false, "error": "VALIDATION_ERROR", "detail": "...", "code": 422}
```

4.4 — Structured Logging

All logs are JSON-structured using the setup in `core/logging.py`. Plain `print()` statements are banned. Every log entry includes: `timestamp`, `request_id`, `module`, `user_id` (if authenticated), `level`, and `message`. This makes log aggregation (CloudWatch, Datadog, Loki) trivial.

05 — Module-to-Module Communication

Modules are vertical slices. They must never be tightly coupled. The following rules govern how modules interact with each other.

5.1 — Synchronous Communication (Direct Service Call)

If Module A needs data from Module B synchronously — it calls Module B's service function. It never touches Module B's repository.

```
# CORRECT — users/service.py calling notifications/service.py
from app.modules.notifications.service import send_welcome_notification

async def create_user(data: UserCreate, db: AsyncSession):
    user = await user_repo.create(db, data)
    await send_welcome_notification(user.id, user.email) # service call
    return user

# WRONG — never do this
from app.modules.notifications.repository import NotificationRepo
```

5.2 — Asynchronous Communication (Worker Tasks)

For heavy operations — emails, push notifications, report generation, bulk processing — the service dispatches an ARQ task. The HTTP request returns immediately. The worker picks up the task from Redis queue.

```
# In service.py — dispatch and return, do not wait
from arq import create_pool
from app.workers.arq_settings import RedisSettings

async def place_order(order_data, db):
    order = await order_repo.create(db, order_data)
    redis = await create_pool(RedisSettings)
    await redis.enqueue_job('send_order_confirmation', order.id)
    return order # returns immediately to client

# In workers/tasks/notification_tasks.py
async def send_order_confirmation(ctx, order_id: int):
    # runs in worker process, separate from main app
    ...
```

06 — Adding a New Module (Step-by-Step)

Every new business domain gets its own module. Follow these steps exactly. Use the scaffold script to avoid missing files.

Step 1 — Run the scaffold script

```
python scripts/create_module.py --name products

# This creates:
# app/modules/products/__init__.py
# app/modules/products/router.py
# app/modules/products/service.py
# app/modules/products/repository.py
# app/modules/products/schemas.py
# app/modules/products/models.py
```

Step 2 — Define your DB model in models.py

Use SQLAlchemy async declarative base. Always inherit from a Base class defined in db/postgres.py. Always include: id, created_at, updated_at, is_deleted (soft delete flag).

```
# modules/products/models.py
from app.db.postgres import Base
from sqlalchemy import Column, Integer, String, Boolean, DateTime
from sqlalchemy.sql import func

class Product(Base):
    __tablename__ = 'products'
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    sku = Column(String, unique=True, nullable=False, index=True)
    is_deleted = Column(Boolean, default=False) # soft delete always
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())
```

Step 3 — Define Pydantic schemas in schemas.py

Separate Request schemas from Response schemas. Never expose the ORM model directly in a route.

```
# modules/products/schemas.py
from pydantic import BaseModel
from typing import Optional

class ProductCreate(BaseModel): # Request
    name: str
    sku: str

class ProductResponse(BaseModel): # Response
    id: int
    name: str
    sku: str
    class Config: from_attributes = True
```

Step 4 — Write repository queries in repository.py

```
# modules/products/repository.py
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from .models import Product
from .schemas import ProductCreate

class ProductRepository:
    async def get_by_id(self, db: AsyncSession, id: int):
        result = await db.execute(select(Product).where(Product.id == id,
                                                         Product.is_deleted == False))
        return result.scalar_one_or_none()

    async def create(self, db: AsyncSession, data: ProductCreate):
        product = Product(**data.model_dump())
        db.add(product)
        await db.commit()
        await db.refresh(product)
        return product

product_repo = ProductRepository()
```

Step 5 — Write business logic in service.py

```
# modules/products/service.py
from sqlalchemy.ext.asyncio import AsyncSession
from .repository import product_repo
from .schemas import ProductCreate
from app.core.exceptions import NotFoundException

async def get_product(db: AsyncSession, id: int):
    product = await product_repo.get_by_id(db, id)
    if not product:
        raise NotFoundException('Product not found')
    return product

async def create_product(db: AsyncSession, data: ProductCreate):
    return await product_repo.create(db, data)
```

Step 6 — Define routes in router.py

```
# modules/products/router.py
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.dependencies import get_db, get_current_user
from app.core.responses import success_response
from . import service
from .schemas import ProductCreate, ProductResponse

router = APIRouter(prefix='/products', tags=['Products'])

@router.get('/{id}', response_model=ProductResponse)
async def get_product(id: int, db: AsyncSession = Depends(get_db),
                     _user = Depends(get_current_user)):
    product = await service.get_product(db, id)
    return success_response(data=product, message='Product fetched')
```

Step 7 — Register router in main.py

```
# app/main.py
from app.modules.products.router import router as products_router
app.include_router(products_router, prefix='/api/v1')
```

Step 8 — Generate and run Alembic migration

```
alembic revision --autogenerate -m 'add_products_table'
alembic upgrade head
```

07 — Project-Specific Module Map (Inventory Example)

The base template modules (auth, users, notifications) never change. For an inventory management project, the following domain modules are added under app/modules/.

organizations/	Multi-tenant root. All data is scoped to an org. RLS enforced at DB level.
products/	Master product catalogue, SKUs, variants (sub-module inside products/).
warehouses/	Locations, zones, bins. Physical structure of storage.
suppliers/	Vendor master data, contact info, lead times.
inventory/	Stock levels per SKU per warehouse. movements/ sub-module for audit trail.
purchase_orders/	Inbound procurement flow. On receipt, dispatches inventory worker task.
sales_orders/	Outbound fulfillment. On ship, dispatches inventory deduction task.
reports/	Read-heavy aggregation. Uses read replica session in db/postgres.py.

Inventory Movement Rule — Critical

Stock levels are never updated directly. Every change in stock — inbound, outbound, adjustment, return — creates an immutable movement record first. Current stock = sum of all movements for that SKU + warehouse. This gives a complete audit trail and makes discrepancy investigation possible.

Movement Flow (PO Receipt Example)

PO marked received → PO service dispatches ARQ task → Worker creates inbound movement record → Stock level updates → Notification fires if reorder point crossed

08 — Background Workers (ARQ)

ARQ is the async-native, Redis-backed task queue used across all Sambhava projects. It runs as a completely separate process — `Dockerfile.worker` — and shares nothing with the main API process except the Redis queue.

Queue Backend	Redis — same instance as auth blacklist, different DB index
Worker Boot	<code>Dockerfile.worker</code> — separate container, separate deploy
Task Location	<code>app/workers/tasks/</code> — one file per domain
Task Naming	snake_case verb-first: <code>send_order_confirmation</code> , <code>generate_report</code>
Retry Policy	<code>max_tries=3</code> , retry on exception, backoff configurable per task
Timeout	Set per task — short tasks 30s, reports 300s
Monitoring	ARQ dashboard or custom health endpoint exposing queue depth

Worker Startup Config

```
# workers/arq_settings.py
from arq.connections import RedisSettings
from app.workers.tasks import notification_tasks, email_tasks

class WorkerSettings:
    functions = [
        notification_tasks.send_push_notification,
        email_tasks.send_order_confirmation,
    ]
    redis_settings = RedisSettings(host='redis', port=6379, database=1)
    max_jobs = 10
    job_timeout = 120
```

09 — Scalability & Hosting Strategy

The monolith is designed to scale horizontally from day one. Below is the progression path as user load grows.

Phase 1 — Early Stage (0–10K users)

Hosting	Single VPS or EC2 — Docker Compose
App	1–2 Gunicorn/Uvicorn workers
DB	Single PostgreSQL instance
Redis	Single Redis instance — auth + queue
Workers	1 ARQ worker container
Load Balancer	Nginx reverse proxy on same server

Phase 2 — Growth Stage (10K–200K users)

Hosting	AWS ECS Fargate or Kubernetes (EKS/GKE)
App	Auto-scaling ECS service — 2 to 10 tasks
DB	RDS PostgreSQL with read replica for reports module
Redis	ElastiCache Redis cluster
Workers	Separate ECS service for ARQ workers — scale independently
CDN	CloudFront for static assets
Secrets	AWS Secrets Manager — never env files in prod

Phase 3 — Scale Stage (200K+ users)

App	Kubernetes with HPA — scale per CPU/memory metrics
DB	Aurora PostgreSQL — auto-scaling storage, multi-AZ
Cache	Redis cluster mode — sharded
Queue	Consider migrating ARQ to SQS for durability
Observability	Datadog or Grafana stack — traces, metrics, logs
CDN	CloudFront + S3 for all media

Stateless Design — Mandatory

The FastAPI app must be completely stateless. No in-memory session storage. No local file writes during request processing. All state lives in PostgreSQL, Redis, or S3. This is what enables horizontal scaling — any request can be served by any app instance.

- Never store user session data in app memory
- Never write files to local disk during a request — use S3
- Never use global mutable variables across requests
- All auth state in Redis
- All file uploads to S3/object storage
- All background state in ARQ/Redis queue

10 — Microservice Extraction Guide

Because each module owns its router, service, repository, schemas, and models with no cross-module repository imports, extracting a module into its own microservice is a structured process, not a rewrite.

Signals That a Module Should Be Extracted

- The module has significantly higher traffic than others (e.g. notifications at 10x)
- The module needs a different tech stack (e.g. reports needs columnar DB)
- The module has different deployment frequency from the rest
- Team ownership is clear and the module is stable — no active refactoring

Extraction Steps

Step 1	Copy the module folder into a new project using the same boilerplate template
Step 2	Replace direct DB models with its own — no shared ORM models between services
Step 3	Replace internal service calls from other modules with HTTP client calls (httpx)
Step 4	Replace ARQ task dispatches from the monolith with HTTP POST or message queue publish
Step 5	Deploy the new service independently with its own DB schema
Step 6	Update the monolith to call the new service via HTTP — keep the interface identical
Step 7	Remove the module from the monolith after validation

Communication Between Services

Synchronous inter-service calls use `httpx` `async` client with `retry` and `timeout`. Asynchronous calls use a message queue (Redis Streams or SQS). Each service has its own database — no shared DB schemas between microservices.

```
# How monolith calls extracted notification service
import httpx

async def send_notification(user_id: int, message: str):
    async with httpx.AsyncClient(timeout=5.0) as client:
        response = await client.post(
            f'{settings.NOTIFICATION_SERVICE_URL}/internal/notify',
            json={'user_id': user_id, 'message': message},
            headers={'X-Internal-Token': settings.INTERNAL_TOKEN}
        )
        response.raise_for_status()
```

11 — Strict Rules & Code Review Checklist

The following rules are enforced during every code review. A PR that violates any of these rules will not be merged regardless of how correct the logic is. These rules exist to protect the team, not restrict it.

Architecture Rules

- **[NEVER]** Import one module's repository into another module
- **[NEVER]** Write business logic inside `router.py`
- **[NEVER]** Write DB queries inside `service.py` — use repository
- **[NEVER]** Call DB session directly inside a router — always use `Depends(get_db)`
- **[NEVER]** Use `print()` for logging — use the structured logger from `core/logging.py`
- **[NEVER]** Hardcode secrets, API keys, or connection strings — use `config.py` + `.env`
- **[NEVER]** Commit a `.env` file — `.env.example` only
- **[NEVER]** Delete DB records — use `is_deleted=True` soft delete
- **[NEVER]** Return SQLAlchemy model objects directly from router — always use response schema
- **[NEVER]** Skip writing a migration after changing a model
- **[ALWAYS]** Use the unified response wrapper from `core/responses.py`
- **[ALWAYS]** Write a test for every new service function
- **[ALWAYS]** Use `async/await` for all DB operations
- **[ALWAYS]** Encrypt sensitive fields before saving using `core/encryption.py`
- **[ALWAYS]** Add the new module's router to `main.py` after creating it

Code Review Gate — PR Checklist

Every PR author must confirm all of the following before requesting review:

Structure	New module follows the exact 5-file structure
Layer purity	No layer violations — router/service/repo responsibilities correct
Schema	Request and Response schemas defined, ORM not exposed
Migration	Alembic migration generated and tested
Soft delete	<code>is_deleted</code> flag present on all new tables
Tests	At least one test per service function
Env vars	Any new config added to <code>.env.example</code> and <code>config.py</code>
Response format	All routes use <code>core/responses.py</code> wrapper

Logging	No print() — structured logger used
No secrets	No hardcoded credentials anywhere in diff

12 — Environment & Configuration

All configuration is managed through a single config.py using pydantic-settings. Values are loaded from environment variables. No module ever reads os.environ directly.

```
# app/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # App
    APP_NAME: str = 'MyApp'
    DEBUG: bool = False
    API_VERSION: str = 'v1'

    # Database
    DATABASE_URL: str
    DATABASE_URL_REPLICA: str = '' # optional read replica

    # Redis
    REDIS_URL: str

    # Auth
    SECRET_KEY: str
    ACCESS_TOKEN_TTL: int = 900 # 15 min
    REFRESH_TOKEN_TTL: int = 604800 # 7 days

    # Encryption
    FIELD_ENCRYPTION_KEY: str # 32-byte AES key, base64-encoded

    class Config:
        env_file = '.env'

settings = Settings()
```

Environment Files

.env	Local development only — never committed to git
.env.example	Template committed to repo — no real values
.env.staging	Managed in CI/CD secrets — never in repo
.env.prod	Loaded from AWS Secrets Manager / Vault — never in repo

13 — Quick Reference Card

Pin this to your workspace. These are the most common commands used daily.

Dev Setup

```
# Clone and start
git clone <repo-url> && cd project-name
cp .env.example .env # fill in your local values
docker-compose up -d # starts postgres + redis
pip install -e '.[dev]'
alembic upgrade head
uvicorn app.main:app --reload

# Start ARQ worker (separate terminal)
arq app.workers.arq_settings.WorkerSettings
```

Common Commands

```
# Scaffold new module
python scripts/create_module.py --name <module_name>

# Create migration after model change
alembic revision --autogenerate -m 'describe_change'
alembic upgrade head

# Run tests
pytest tests/ -v

# Rollback last migration
alembic downgrade -1

# Check pending migrations
alembic current
alembic history
```

Architecture Reminder

Router → Service → Repository → Database

Service calls other Services | Heavy work goes to Workers | Sensitive fields use Encryption